Can AI-generated data validation rules be trusted alongside expert controls?

MSc Artificial Intelligence thesis proposal

Tomash Oosterbaan
University of Groningen | Avanade and Rabobank
2 November 2026

1 Problem and research objective

Data teams check incoming data before it is used in reports, analyses or other systems. These checks range from simple technical controls, such as required fields and valid codes, to business rules that relate several fields to each other. At Rabobank, this work includes technical validation, business rules, monitoring thresholds and the manual investigation of exceptions [1]. Banks must also be able to show that such controls exist and work: the Basel Committee's principles for risk data (BCBS 239) and the European Central Bank's 2024 guide treat data quality and its controls as part of sound risk management [2, 3].

Most of these controls are validation rules. A validation rule is a condition that every record must meet, for example "total assets must equal total liabilities plus equity" in a balance sheet. When a record does not meet the condition, the rule raises an alert, and a person investigates whether the data is wrong. Writing good rules requires knowledge of the data and of the business, and maintaining them takes time.

A language model can write candidate validation rules from a description of the data within seconds, and data-quality tools have started to offer this [4]. Some of these rules are useful. Others apply a wrong business assumption, repeat a control that already exists, or raise alerts on records that are correct. A rule such as "the loan amount must be below five times the income" may look reasonable, but it is not a valid data-quality rule unless the business has such a policy. In addition, the same model can write different code when asked the same question twice [5], and rules that are discovered automatically from data often turn out to be false [6].

For a team that already maintains many checks, three questions therefore matter. Does a generated rule do what it is supposed to do? Does it catch errors that the existing controls miss, and how many extra alerts does that cost? And would the rule have behaved well on past data, so that the team can decide whether it may stop data automatically, may only warn, or should only be monitored?

This thesis studies these three questions. It measures how correctly and consistently language models implement documented expert rules, how much they add beyond independent expert controls, which method best selects a small set of useful rules, and how the selected rules behave when they are run on historical data in the order in which that data arrived. The study uses public datasets and publicly documented rules; no Rabobank data is required. A software workbench will implement the procedure, and a small data pipeline will show how its results can be used in practice.

2 Pilot work and change of research direction

The direction of this thesis follows from two pilot studies carried out in September and October 2026 [7] and a feasibility check carried out in November 2026 [8]. Both pilots used controlled error injection: values were changed on purpose in a copy of real data, and every change was recorded. Because the location of each error is known, it is possible to count which errors a control finds,

which it misses and how many alerts it raises on data that was not changed. The feasibility check instead ran rules on real reports without changing them. The figures below are internal pilot results, not independently published findings.

2.1 First pilot: choosing an error detector

The project began with a question about choosing error detectors for tables of data. Error detectors are methods that flag suspicious values without a separate rule being written for every possible error. For example, the CDC method predicts each value from the other columns of the same record and flags values that do not fit the prediction [9]. Earlier benchmark studies had shown large differences between such methods, and had shown that the effort needed to review their alerts is an important part of their cost [10, 11]. Jung et al. showed that the way errors are spread over the data can change how well a detector appears to perform [12].

The pilot therefore separated the type of change made to a value from the mechanism that decides which values are affected. Three mechanisms were tested. Erroneous completely at random (ECAR) means that every value has the same chance of being wrong. Erroneous at random (EAR) means that the chance depends on another recorded field, for example the source system that delivered the record. Erroneous not at random (ENAR) means that the chance depends on the value itself, for example when larger amounts are more likely to contain an error. The pilot applied missing values, invalid types and codes, amounts multiplied by ten and plausible substitutions to three public tables: credit-card clients, bank marketing and wine quality [13, 14, 15]. It compared rule-based and statistical controls with seven detectors, including CDC and Raha [9, 16], and measured how much of their added value remains when a reviewer can check only a limited number of alerts.

The error mechanism changed how many errors a detector found, but not which detector was best. The existing rules already found every missing value, invalid type and invalid code. On harder errors, the strongest additional detector found 11 to 21 of 40 changed values that the rules, the statistical checks and a simple prediction model missed, but it produced 520 to 900 extra alerts per batch of 800 records. With a budget of 20 reviews per 800 records, the extra alerts compete with the alerts of the existing controls for the reviewer's attention. The net gain, the number of extra errors found within those 20 reviews, was on average about 0.4 errors on the bank-marketing data, zero on the credit-card data and negative on the wine data. A negative gain means that the detector's alerts pushed useful alerts of the existing controls out of the review.

2.2 Why the research direction changed

The first pilot was not continued for four reasons. First, the mechanism changed the size of the effect but not the choice of detector, so there was little to select. Second, the question of what an extra detector adds when review time is limited had already been studied [10, 11]. Third, the public tables contain no record of real errors, so the pilot could only show that a detector recovers changes that we made ourselves. Fourth, the tables come without expert rules, so the pilot could not ask what an automatic method adds beyond the rules a real data team already has. The second pilot therefore moved to data that comes with an independent, published set of expert rules.

2.3 Second pilot: rules written by language models

The second pilot used financial reports that US listed companies file with the Securities and Exchange Commission (SEC). The Data Quality Committee (DQC) of XBRL US, an industry body, publishes validation rules for these reports together with their code and official test cases

[17]. Two small open-weight language models, Qwen2.5-Coder-7B and Qwen3-8B [18, 19], wrote rules in three ways: from the written description of a DQC rule, from an example report, and for types of error that the existing controls missed.

Three results stand out. First, reproducing the behaviour of the official rules was unreliable. For rules published before the models were released, the generated rules produced about 41% of the messages that the official test cases expect; for rules published afterwards, only 3% and 13% for the two models. Generating the same rule again often gave a very different result. Second, the generated rule sets were too noisy to use as a whole. The existing controls detected 27 of 83 injected errors; the best generated rule set detected six further errors, but raised on average 458 alerts per report that had never been corrected, against 1.8 for the DQC rules. Third, real corrections proved hard to use as ground truth. Of 20 numbers that companies had later corrected, only 9 were clearly wrong in the original report, and 7 of these were accounting restatements rather than data errors.

2.4 Follow-up: judging individual rules

A follow-up experiment looked at the causes and at individual rules instead of complete rule sets. When the model received the official code of nine rules it had failed to reproduce, it fixed three. Most of the remaining failures came from missing domain knowledge, such as information that is only available in the XBRL dictionary of reporting concepts, rather than from missing documentation. When individual generated rules were selected only if they detected errors that the existing controls missed, within a limit of one alert per report, one rule remained. It detected three additional injected errors at about 0.2 alerts per report. A generic filter that ignored the existing controls needed about twice as many alerts for the same gain. This is a small exploratory result and does not establish that the method is better. It does show that useful rules can be hidden inside noisy rule sets, and that selecting them relative to the existing controls is worth testing with stronger models, more data and a proper held-out evaluation.

2.5 Feasibility check: testing rules on historical data

The last step tested whether rules can be evaluated on historical data in the order in which it arrived, as a data pipeline would see it. Twenty-nine SEC reports filed between February 2023 and March 2025 were processed in filing order. The official DQC rules and the generated rules from the second pilot were run on each report, and every alert was linked to its report, its rule, the rule version and the date from which the rule existed. This worked for 98% of the alerts, and the official rules took about two minutes per report. Three results matter for the design. First, 46% of the DQC alerts came from rules that were published only after the report had been filed, so a test with today's rules overstates what a team could have caught at the time. Second, reports from early 2023 need an older release of the DQC rules. Third, none of the 13 checked corrections was flagged by any rule, expert or generated; most of them were accounting restatements, which validation rules are not designed to catch [8].

A separate short study looked at a different idea. The European Banking Authority (EBA) publishes thousands of validation rules that the supervisory reports of European banks must pass. Sometimes a rule itself turns out to be wrong, and the EBA then switches it off. The study tested whether a language model could have predicted which rules would later be switched off. The model ranked a faulty rule above a correct one in 64% of the cases, where 50% would be chance, but simply ranking rules by the length of their formula did better (71%), and within a single reporting table the model was close to chance (53%). This direction was therefore not selected [7].

2.6 What the pilot work shows

Table 1 summarises how the pilot results shape this proposal. Together they suggest that there is something to gain: a small number of generated rules can add detections at a low alert cost. They also show that this gain only becomes visible when rules are judged one by one, relative to the controls that already exist, and when they are tested in a way that matches how they would be used. So far, the gain has only been shown on injected errors and for a single rule. Whether generated rules also find real errors that the existing controls miss is still open; that is the core of this thesis.

Pilot result	Consequence for this thesis
Simple rules already caught all format and code errors; extra detectors mainly added alerts.	Every method is compared with the existing controls, and alerts are always reported next to detections (RQ2).
Generated rules reproduced older official rules much better than newer ones.	Rules are split by publication date relative to the model's release date (RQ1).
Most failures came from missing domain knowledge, not missing documentation.	One experimental condition adds domain knowledge to the model's input (RQ1).
Whole generated rule sets were too noisy, but one selected rule added detections at 0.2 alerts per report.	Selecting rules relative to the existing controls is the main proposed method (RQ2).
Most real corrections were accounting restatements, and none of the 13 checked corrections was flagged.	A larger, carefully reviewed set of real errors is built, weighted towards likely data errors (RQ3).
Almost half of today's alerts came from rules that did not yet exist at filing time.	The historical test distinguishes rules available at the time from rules developed later (RQ3).

Table 1: Pilot results and their consequences for the proposed research.

3 Related research and proposed contribution

This section describes five lines of earlier work. For each line it explains what the studies do, how they judge their results, and what they leave open for this thesis. Table 2 at the end of the section compares the most relevant studies.

3.1 Detecting errors and the cost of reviewing alerts

Abedjan et al. ran several error detection tools on real datasets in which the errors were known. Each tool found only part of the errors, and combining tools found more errors but also produced more alerts that a person had to verify [10]. REIN later compared a large number of detection and cleaning methods on the same datasets [11]. The study by Abedjan et al. in particular shows that a new check should be judged by what it adds to existing checks and by the review effort it creates, not by its detections alone. Both studies evaluate data-driven detectors, however, not rules written by language models.

3.2 Language models that write data-quality rules

Alexander et al. asked language models to write data validation tests, small programs that check a dataset, and compared them with a test suite written by one data scientist. They repeated each setup five times to measure how consistent the generated tests were [20]. Akella et al. combine

statistical screening of the data with retrieved domain information: the model proposes rules, translates them into executable code, and presents each rule on a readable card for review [21].

LeDQeR asks a model to generate candidate rules for Great Expectations, an open-source tool in which checks are written as "expectations" about a table, starting from an example of an erroneous record. Four filters then remove rules that do not run, that are wrong on labelled data, that only fit a single record, or that repeat another rule. The rules are scored on benchmark tables in which every error is labelled [22]. AuDQ uses several cooperating "agents", language-model programs that plan steps, call tools and check their own output, to discover rules. The authors also describe how rules can be versioned and traced to their origin, and they let people rate the rules. They note that their datasets have no reference rule set to compare against [23].

Two master's theses address the same task. Schneider guides the model with templates that fix the shape of a rule and compares the output with five to ten expert "golden rules" per dataset, judging by hand whether each golden rule is covered [24]. Siyam compares several commercial models and prompting strategies for generating data-quality rules [25].

These studies establish that rule generation, filtering and human review with language models are existing research directions. They judge the generated rules against labelled benchmark errors, human ratings or a small set of reference rules. None of them compares generated rules with a large, independently maintained rule set using that rule set's own executable tests, and none asks what a generated rule adds beyond the controls that are already running.

3.3 Selecting a small set of useful checks

Generating many candidate checks is easy; choosing the few that are worth keeping is harder. SPADE addresses this for applications built on language models. It derives many automatic checks on the model's output from the changes developers made to their prompts, and then selects a small subset that still catches the failures developers have labelled, while keeping the share of false failures below a chosen limit [26]. Auto-Validate learns patterns for text columns, such as the format of a date or a product code, from a large collection of tables, and chooses patterns that keep the expected number of false alarms low [27]. LeDQeR's last filter removes a generated rule when another generated rule already covers it [22].

These methods judge candidates on their own or against each other. None of them selects relative to an existing set of expert controls, so they may keep checks that only repeat what those controls already catch. This is the gap that the main method of this thesis addresses.

3.4 Checking recurring data and testing on history

Many data sources deliver new data in recurring batches. Redyuk et al. learn from statistics of earlier batches what a normal batch looks like and flag new batches that deviate [28]. Auto-Validate-by-History, developed at Microsoft, uses the history of earlier runs of a recurring pipeline to choose quality constraints with a target false-alarm rate, and was evaluated on about 2,000 production pipelines [29]. Both use history to build and test statistical checks; neither tests rules written by language models against expert rules.

Modern data platforms, often called lakehouses, store data as files and add database features such as versioned tables [30, 31]. In such platforms, new data can first be written to a separate area, checked, and only then published. Tagliabue and Greco argue that AI agents should change a data platform only through such isolated and checked steps [32]. Benchmarks show why care is needed: in ELT-Bench, the best AI agent at the time of publication built only 11.3% of the required data models correctly [33]. In banking, testing a model on past data before trusting it, known as backtesting, has long been standard practice for risk models [34]. We did not find work

that backtests generated data-quality rules in this way, or that uses such a test to decide whether a rule may stop data.

3.5 Evaluating code written by language models

Three effects complicate the evaluation of generated rules. Language models perform worse on specialised programming languages with little public code, such as the rule language used for financial reports [35]. Their output can differ between repeated runs [5]. And public rules may have been part of their training data, so a test on such a rule may measure memory rather than skill; benchmarks for generated code therefore separate test items by publication date [36]. For the OLMo models, whose training data is public, it is also possible to search the training data directly for a given text [37, 38].

3.6 Proposed contribution

Table 2 summarises the comparison. Systems that generate data-quality rules with language models also exist commercially and have been patented [4, 39]; the contribution of this thesis is therefore not the idea of such a system. The anticipated contribution is an empirical evaluation with four parts: (1) generated rules are scored against independently maintained expert rules, using those rules' own tests and controlling for rules the model may have seen; (2) a method that selects generated rules by what they add beyond the existing expert controls is compared with alternative selection methods at fixed alert budgets; (3) the selected rules are tested on historical data in the order in which it arrived, using reviewed real corrections; and (4) a reviewed set of real errors in financial reports is produced. In short, the thesis treats data-quality rules the way banks treat risk models: before a rule is trusted, it is tested against an independent reference and on historical data. This claim is provisional and will be reassessed against literature published during the thesis.

Study	What it does	How results are judged	Accounts for existing expert controls	Tested on history
Alexander et al. [20]	Generates data tests	One suite by one data scientist; five repeats	No	No
Akella et al. [21]	Generates rules and code with retrieved knowledge	Benchmark data; rule cards	No	No
LeDQeR [22]	Generates and filters rules	Labelled benchmark errors	No	No
AuDQ [23]	Agents discover rules; provenance	Human ratings	No	No
Schneider [24]	Template-guided generation	5 to 10 expert rules per dataset, judged by hand	No	No
SPADE [26]	Selects checks on LLM output	Labelled failures; false-failure limit	No	No
Auto-Validate-by-History [29]	Learns statistical constraints	Past pipeline runs	No	Yes
This proposal	Generates, selects and backtests rules	Official expert tests; injected errors; reviewed real corrections	Yes	Yes

Table 2: Comparison with the most relevant earlier work, as far as reported in the cited sources.

4 Research questions

Main research question: *Under what conditions can validation rules generated by open-weight language models reproduce expert-defined checks, provide additional error detection beyond existing expert controls within a practical alert budget, and be used safely in a data pipeline?*

RQ1. How accurately and consistently can language models implement documented expert rules, and what is the effect of adding domain knowledge and of letting the model test its rule on development examples?

RQ2. How much additional error detection do generated rules provide beyond the existing controls, how do five selection methods compare at fixed alert budgets, and can the evidence from this selection decide whether a rule should block data, only warn, or only be monitored?

RQ3. When the selected rules are run on historical financial reports in the order in which they were filed, how many reviewed real errors do they detect, how many alerts do they raise, how stable are they over time, and does the chosen level of action still hold on later data?

RQ1 tests whether a model can reproduce what experts already wrote; it follows from the reproduction and failure results of the second pilot. RQ2 tests whether generated rules can add something that the experts did not write; it follows from the follow-up experiment, in which one selected rule added detections at a low alert cost. RQ3 tests whether those results still hold on real data over time; it follows from the feasibility check.

5 Data sources and reference material

5.1 What "independent expert rules" means

Each public dataset in this study comes with validation rules that were written and published by an organisation outside this project. These organisations maintain the rules for real reporting, document them, and in most cases also publish code or test cases that show what each rule should report (Table 3). Because neither the author nor the language models wrote these rules, they can serve as an independent reference: a generated rule can be compared with them, and they form the existing controls against which any additional value is measured.

Organisation	Who they are	What they publish
XBRL US Data Quality Committee (DQC)	Industry committee for US digital financial reporting	About 200 rules for SEC reports, with code and official test cases [17]
XBRL International	International standards body for XBRL	Data-quality rules for European company reports, with test cases and expected results [43, 44]
FFIEC and CFPB	US financial regulatory bodies	"Edits" for mortgage reporting, with open-source validation software and tests [45, 46]
GLEIF	Operator of the global register of legal entities	207 written data-quality checks, without code [47]

Table 3: Organisations that wrote the expert rules used in this study.

5.2 Datasets

D1: US company reports and DQC rules. The SEC is the US regulator that receives the reports of listed companies. These reports are filed in XBRL, the eXtensible Business Reporting Language. In XBRL every number is a separate fact with a standard concept name, a reporting date or period, a unit and a level of rounding, for example the concept `Assets` with a value of 340,000,000 US dollars on 31 December 2014. A comparison between facts must therefore respect their accounting meaning, not only their values.

Rule DQC_0004, for example, checks that assets equal liabilities plus equity, and its documentation gives the message "Assets with a value of 340,000,000 is not equal to the total of Liabilities and Equity with a value of 350,000,000" [17]. The DQC rules are written in the rule language XULE and run with the open-source XBRL processor Arelle [17, 40]. Of these rules, 153 have official test cases; 38 were first released after 29 April 2025, the release date of Qwen3, which makes it possible to test rules that a model released before that date cannot have seen. The SEC Financial Statement Data Sets show which reports were later amended: from 2023 to mid-2025, 1,294 of 68,875 reports were followed by a corrected version [41]. D1 is the main dataset, because it has the most complete reference and the pilot software already runs on it.

D2: European company reports and the ESEF rules. ESEF, the European Single Electronic Format, is the format in which issuers listed on EU regulated markets publish their annual financial reports [42]. Consolidated statements prepared under International Financial Reporting Standards (IFRS) are tagged in this format with Inline XBRL. XBRL International publishes data-quality rules for these reports that are derived from the DQC rules [43]. Of the 25 rules in the current set, 22 have official test reports, and an official file lists the expected results of the rules on 1,739 real European reports from 2022 [44]. D2 tests whether results from the US setting also hold under different accounting standards and for different companies.

D3: US mortgage applications and HMDA edits. Under the Home Mortgage Disclosure Act (HMDA), US lenders above certain size thresholds report data on the mortgage applications they receive. The regulators publish "edits", validation rules described in plain language, of four kinds: syntax edits check the file format, validity edits check that fields contain allowed values, quality edits flag unusual values that the lender must confirm, and macro quality edits look at a file as a whole [45]. Quality edit Q604, for example, checks that the reported county exists in the census list for the reporting year. The regulator's own validation software is open source and contains 318 edit codes and 330 test files [46]. The public loan data has been modified for privacy and has already passed the filing controls, so only suitable fields and controlled error injection will be used. D3 tests a source that is closer to the ordinary tables in a bank's data platform.

D4: The GLEIF register of legal entities. The Global Legal Entity Identifier Foundation (GLEIF) maintains the register of Legal Entity Identifiers (LEIs). An LEI identifies a legal entity, such as a company, across countries, and banks use it to identify the parties they deal with. GLEIF publishes 207 data-quality checks, each written as a precondition, a condition and an expected outcome; a check can pass, fail or not apply [47]. Check C000020, for example, fails when a legal name is "n/a" or consists only of punctuation. GLEIF does not publish code or a result for each record, so a small test suite must be built (Section 5.4). D4 is optional if time is limited.

D5: Reviewed real errors in SEC reports. Companies sometimes file a corrected version of a report after the original has been published. Comparing the two versions shows which numbers changed. A changed number, however, is not always a data error in the original report. Table 4 shows the three main kinds of change; only the first is the kind of error that a validation rule can be expected to catch. In the second pilot, for example, a company's total assets changed from 7,170,767 to 4,670,767 US dollars because a receivable had been recorded incorrectly in its accounts. The original report correctly showed the accounts as they stood at the time, so no rule applied to that report could have flagged the number. D5 is a reviewed set in which each changed

number is assigned to one of these kinds (Section 5.4).

Kind of change	Example	Can a validation rule detect it in the original report?
Data error	A typing error, a reversed sign, or a number reported for the wrong period or under the wrong concept	Yes, in principle
Accounting restatement	The company later finds that a receivable was recorded incorrectly and revises its accounts	Usually not: the original report matched the accounts at the time
Relabelling only	The same number is moved to a different concept	Usually not an error

Table 4: Kinds of change between an original report and its corrected version.

5.3 What counts as the correct answer

To decide whether a rule is good, we need to know what the correct answer is. This reference is often called the ground truth. It differs per research question, because each question asks something different (Table 5).

For RQ1, the question is whether a generated rule behaves like the expert rule, so the expert rule itself is the reference. The DQC publishes, for each rule, test reports together with the exact messages that the rule must produce for them. For a rule that compares total assets with liabilities plus equity, such a test report would contain a balance sheet in which the two totals differ, and the expected output would be the corresponding message. A generated version of the rule is correct when it produces the same messages on the same test reports.

For RQ2, the question is whether a generated rule finds errors that the expert rules miss. The expert rules cannot be the reference here, because they miss errors themselves: in the second pilot, all existing controls together detected 27 of 83 injected errors. Instead, errors are injected into a copy of the data. If, for example, the sign of a company's reported revenue is reversed in the copy, we know exactly which number is wrong and can check which rules flag it.

For RQ3, the question is whether the rules also find real errors. Here the reference is the reviewed set of real corrections (D5), complemented by a random sample of alerts that is checked by hand.

Question	Reference	Main limitation
RQ1: does the generated rule behave like the expert rule?	Official test reports with expected messages (D1 to D3); constructed test records (D4)	Shows what the rule should report, not whether the data is wrong
RQ2: what does the generated rule add?	Injected errors with known locations	Exact, but the errors are not real
RQ3: what would the rule have done on real data?	Reviewed corrections (D5) and a hand-checked sample of alerts	Small; a report that was never corrected is not proven correct

Table 5: Reference used for each research question.

The reference for RQ3 has two limitations. First, a report that was never corrected is not necessarily free of errors. An alert on such a report may point to a real problem that nobody corrected, so it is not automatically a false alarm. Alerts on these reports are therefore treated as a measure of review work, and the share of false alarms is estimated from the hand-checked

sample. Second, as Table 4 shows, a correction is not always an error that could have been seen in the original report. Each reviewed item therefore records whether the problem was visible in the original report.

5.4 Reference material built during the project

Three references do not exist yet and will be built during the project.

Reviewed real errors (D5). The target is 300 changed numbers, spread over companies and over kinds of change, with extra weight on changes that are more often data errors, such as a reversed sign, a factor of 1,000 or a shifted reporting period. For each number, a reviewer opens the original and the corrected report, reads the company's explanation of the correction, and records three things: whether the original number was wrong, which kind of change it is (Table 4), and whether the problem was visible in the original report. The pilot used this procedure for 20 numbers, with labels suggested by a language model and checked by the author. In the thesis, the author labels all 300 numbers without such suggestions, following a written guide with examples. A second reviewer, preferably a professional with accounting or XBRL experience, labels the same 100 numbers independently. Agreement is measured with Cohen's kappa, a standard measure that corrects for agreement by chance [48], and disagreements are reported. If no independent reviewer is available, the study will report the lack of an external gold standard and will not present the author's labels as verified fact.

Test records for the GLEIF checks (D4). This reference is needed only for RQ1 on the GLEIF data. GLEIF publishes its checks as text, but no code and no examples of correct outcomes, so there is nothing to compare an AI-written version of a check with. For 40 to 60 checks, we therefore write three small artificial records per check, directly from the published text: one that should pass, one that should fail and one to which the check does not apply. Table 6 shows this for check C000020, which states that the legal name of an active entity may not be a placeholder such as "n/a". A second person reviews the records.

Record	Registration status	Legal name	Expected outcome of C000020
1	Issued	Example Holding B.V.	Pass
2	Issued	n/a	Fail
3	Annulled	n/a	Not applicable

Table 6: Constructed test records for GLEIF check C000020.

Hand-checked alert sample (RQ3). From the alerts raised in the historical test on reports that were never corrected, a random sample of 200 is checked by hand with the same guide. This shows how many of these alerts point to real problems and how many are false alarms.

6 Experimental design

6.1 RQ1: Reproducing expert rules

For each rule with a reference, a model will write an executable implementation under three input conditions: (A) the written description of the rule and the necessary data structure; (B) the same information plus relevant domain knowledge, such as whether an amount is normally positive, whether it describes a moment or a period, and which amounts add up to which total; and (C) condition B with permission to run the rule on development examples and repair it. The official test outcomes will never be given to the model. Each task will be repeated five times with fixed

generation settings to measure variation. The generated messages will be compared with those of the official implementation, reporting the share of expected messages found (recall), the share of produced messages that are correct (precision), exact agreement per test case, the share of code that does not run, and the variation between repeats.

A complication is that the DQC rules are public: their code has been available on GitHub for years. A model that reproduces a rule correctly may therefore have seen that rule during its training, in which case the result measures memory rather than skill. Two checks address this. First, rules are split by publication date. A rule published after a model was released cannot have been part of its training data, so it is the fair test of whether the model can write a rule it has not seen. The second pilot already pointed in this direction: the generated rules produced about 41% of the expected messages for older rules, against 3% and 13% for newer rules. Second, for most models the training data is not public, so it is impossible to check what they have seen. The OLMo models of the Allen Institute for AI are an exception: their training data is published, together with a tool that finds exact matches between a given text and that data [37, 38]. For OLMo, we can therefore check directly whether the text of a DQC rule was part of the training data. This is a small additional check, not a main part of the study.

Finally, a language model will judge, by reading only, whether each generated rule is equivalent to the expert rule, as in earlier work that judged coverage by reading [24]. Comparing this judgement with the execution results shows how often a rule that looks correct does not behave correctly.

6.2 RQ2: Additional detection and rule selection

RQ2 asks whether generated rules can find errors that the existing controls miss. The experiment has four steps.

First, the existing controls are fixed. They consist of the official expert rules, basic format checks and inexpensive statistical checks learned from other reports, for example a check for an unusual sign or a sudden jump between periods.

Second, errors are injected into a copy of reports that are not known to contain errors. The error types are based on what goes wrong in practice: a reversed sign, a value that is 1,000 times too large because of a wrong unit, a changed digit, a wrong concept or code, swapped reporting periods and a missing value. Where real corrections are available, they determine how large the injected changes are. Running the existing controls on the changed copy shows which error types they catch and which they miss. In the second pilot, for example, the DQC rules caught between 17 and 64% of most error types, but none of the removed values. Each injected change is recorded at the level of the affected fact or field, and results on injected errors are kept separate from conclusions about real error rates.

Third, models generate candidate rules for the error types that the existing controls miss. Candidates are generated on development material and evaluated on companies that were not used during development.

Fourth, the candidates are selected. Five selection methods are compared, using the same candidate pool, the same observations and the same budget: (1) keep every rule that runs; (2) a generic filter based on validity and alert volume, in the style of LeDQeR [22]; (3) a compact selection that balances coverage and false alarms, inspired by SPADE [26]; (4) a language model that rates the plausibility of each rule; and (5) an expert-aware selection that keeps only rules detecting errors that the existing controls miss, while also considering alert volume and repeatability. The fifth method is the main proposed method; its scoring and thresholds will be registered before the final evaluation.

The primary outcome is the number of additional detections at fixed average alert budgets of 0.5,

1 and 2 alerts per report. The budget makes the practical trade-off explicit: a rule that detects ten extra errors but raises 450 alerts per report is of no use to a team that can review one alert per report. Secondary outcomes include recall, precision where clean labels exist, alerts on reports that were never corrected, runtime, the share of generated rules that fail, and stability across repeats. From the same evidence, each selected rule receives a proposed level of action: block (hold the data), warn (let the data pass with an alert) or monitor (record only). The thresholds for these levels are fixed in advance; whether the levels are reliable is tested in RQ3, and a person always takes the final decision.

This procedure also works outside the experiment. When a team onboards a new data source, it does not know which real errors the data contains, and it does not need to. What it needs to know is which kinds of error its current controls cannot see. Error injection answers that question in the same way that a fire drill tests an alarm without waiting for a real fire: the team inserts typical errors into a copy of its own data, sees which kinds slip through, lets a model propose rules for exactly those kinds, and checks how many alerts each proposal raises on the real, unchanged data. The weakness is that an error type that is not on the list is not tested, which is why RQ3 checks the results against real errors.

If the existing controls already catch an error type, generated rules have nothing to add for that type, and the study will report this as a result. The pilots show that this differs per error type. In the first pilot, ordinary rules caught every missing value, wrong data type and invalid code, so there was nothing to add. In the second pilot, subtler errors were often missed: the existing controls detected 27 of 83 injected errors and none of the removed values.

6.3 RQ3: Testing selected rules on historical data

RQ3 asks how the selected rules would have behaved if they had been in use in the past. The test works like a replay. About 2,000 real SEC reports from 2023 to 2026 are processed week by week, in the order in which they were filed, and the expert rules and the selected generated rules are run on each week's reports as if they had been active at that time. Every alert is stored with its report, its rule, the rule version and the date from which the rule existed. Based on the feasibility check, the official rules take about two minutes per report, or roughly 70 hours of computing time in total.

An example shows what this test reveals. Suppose a generated rule states that a company's quarterly revenue cannot be negative. In the replay, it might flag a few reports per year, some of which were later corrected for exactly that number. Such a rule could reasonably be allowed to block data. If it instead flagged dozens of reports per week that were never corrected, it would cost too much review time to block anything, and it should at most warn or be monitored. Only a test on real data over time shows which of these two cases applies.

The replay is run in two forms, because they answer different questions:

1. Historical stress test. The rules as they exist today are applied to past reports. This shows how a rule behaves on real data from different periods.
2. Time-bound backtest. Only information that was available at the time is used: the version of the DQC rules that was in force on each filing date, generated rules that were selected using earlier data only, and a model that was released before the replayed period. This shows what a team could really have caught at the time. The difference matters: in the feasibility check, 46% of today's DQC alerts on reports from 2023 to 2025 came from rules that did not yet exist when those reports were filed. The time-bound form will therefore start in May 2025, after the release of Qwen3. All DQC releases are public [17]; if older releases do not run with the current software, the current rule code will be filtered by release date and this will be reported as a limitation.

Several rule sets are compared in the replay: the expert rules alone; the expert rules plus all generated rules; and the expert rules plus the rules kept by each of the five selection methods. For each rule set, five things are measured:

1. Real errors found: how many of the reviewed errors in D5 would have been flagged in the original report.
2. Review work: how many alerts per week the rule set raises on reports that were never corrected.
3. False alarms: what share of these alerts is a false alarm, estimated from the hand-checked sample of 200 alerts.
4. Stability: whether a rule keeps the same behaviour from quarter to quarter.
5. Level of action: whether a level chosen on earlier data, for example "block" based on 2023 and 2024, is still justified by the results in 2025 and 2026.

Recent reports have had less time to be corrected, so these results are reported per period.

6.4 Professional review of the evidence

The platform (Section 8) summarises each generated rule on an evidence card: one page that tells a reviewer what the rule does and how it performed in the experiments. Table 7 shows what such a card contains. The values in the table are invented for illustration and are not results.

Field	Example content (illustrative only)
Rule in plain words	Quarterly revenue must not be negative, except for the concepts on an exclusion list
Origin	Generated by a local model for the error type "reversed sign", with domain knowledge added to its input
Added beyond existing controls	Catches 14 of 20 injected sign errors in revenue that the expert rules miss
Alerts in the historical test	0.3 per week on reports that were never corrected
Real errors in the historical test	2 reviewed sign errors flagged before the company corrected them
Stability	Same result in 5 of 5 generations and in every quarter
Proposed level of action	Warn
Decision	Accept, reject or revise, with the reviewer's reason

Table 7: Contents of an evidence card. All values are invented for illustration.

Three to five data-quality or accounting professionals, for example consultants at Avanade, will each review about 30 such cards. For each card they record whether they would accept, reject or revise the rule, at which level of action, and how long the decision took. Their decisions are compared with the levels proposed by the method. This shows whether the evidence is understandable and useful to the people who would approve rules in practice. Reviewer recruitment, ethics approval from the faculty and permission to release derived labels will be arranged before this stage. This part is an extension if reviewers are not available in time.

6.5 Protocol, data splits and statistics

Three measures keep the evaluation fair. First, all choices that could influence the outcome, such as the alert budgets, the selection thresholds and the outcome measures, are written down in a

dated protocol before the final evaluation is run. In the pilots, some limits were chosen after the first results had been seen; the protocol prevents this. Second, rules are selected on one group of companies and evaluated on another group and, where relevant, on a later period. A rule that only works because it was tuned to the reports of particular companies will then fail the evaluation. Third, every result is reported with a 95% confidence interval, a range that shows how certain the number is given the size of the sample, obtained by resampling over companies or rules. Methods that run on the same data are compared pairwise, and all runs are reported, including failed runs and results that go against the expectations.

7 Models, baselines and reproducibility

The project will use open-weight language models that can be run on controlled infrastructure (Table 8). Such models can be downloaded and run locally, so no data has to be sent to an external service, the exact model version can be stored, and the release date is known, which the contamination analysis requires. The larger experiments will run on the university's computing cluster Hábrók, where software runs in Apptainer containers [49]; the small models remain usable on an ordinary computer. Self-hosting avoids a mandatory external inference service, although computing and maintenance still have costs.

Model	Released	Role
Qwen2.5-Coder 7B and 32B [18]	2024	Specialised in code; the 7B version provides continuity with the pilots.
Qwen3 8B and 32B [19]	April 2025	General model; the 8B version provides continuity with the pilots.
gpt-oss 20B, optionally 120B [50]	August 2025	Second model family, to test whether results depend on one developer.
OLMo 2 7B and 32B [37]	2024–2025	Public training data, which allows a direct contamination check.

Table 8: Planned models. "B" stands for billions of parameters, a rough measure of model size.

Model versions, generation settings and prompts will be fixed and documented. All selection methods see the same generated candidate rules and the same development labels; only the use of the existing expert controls differs. This prevents a stronger generator or extra test knowledge from being mistaken for a better selection method. As a secondary baseline, a language model will also inspect reports directly and point out suspicious facts, without producing reusable rules. This makes the difference between one-time model judgements and reusable, executable checks explicit.

8 Engineering artefact and organisational use

The engineering deliverable is a platform for testing data-quality controls before they go live, whether they were written by people or by a language model. It is built from the pilot software and consists of eight components (Table 9); Figure 1 shows how they work together.

The pipeline is built with Apache Spark and Delta Lake, the open-source technologies on which platforms such as Databricks are built [51, 31]. New reports arrive in a staging area, the official rules run through Arelle, the approved generated rules run next, and each batch is then published, published with warnings, or held back for review, according to the levels of the rules that fired. The audit log is kept separately from the data tables, because older table versions are removed

Component	What it does
Rule register	Stores every rule, expert-written or generated, as versioned code with its origin, the date from which it was available and its level of action.
Stress-test engine	Injects realistic errors into a safe copy of the data and shows, per error type, which existing controls catch it: a map of blind spots.
Rule studio	Lets a local language model propose rules for the blind spots, using domain knowledge, and runs and repairs them in a restricted environment.
Selection	Keeps only rules that add detections beyond the existing controls within the alert budget, and proposes a level of action.
Backtest engine	Replays historical data in time order and reports what each rule would have caught and how many alerts it would have raised.
Quality gate	Runs the approved rules in a data pipeline and publishes each batch, publishes it with warnings, or holds it for review.
Review screen	Shows the blind-spot map and the evidence cards, and records accept, reject or revise decisions.
Audit log	Stores every decision with its evidence in an append-only log, so that it can be shown to an auditor.

Table 9: Components of the platform.

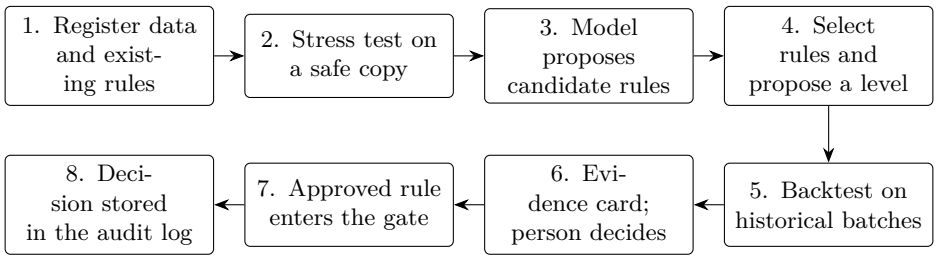


Figure 1: How the platform is used. It never adopts a rule by itself and never changes data.

after a retention period, by default seven days [52]. Generated code runs without production credentials, and the platform never adopts a rule or changes data by itself.

A typical use would look as follows. A data team at a bank starts using a new external source, for example the published financial statements of its business clients. The team registers the source and its current checks. The stress test shows that reversed signs and values reported in the wrong unit slip through. A local model proposes a handful of rules for these gaps, and the selection keeps two of them. The backtest shows, for each of the two, how many real errors it would have caught in the past two years and how many alerts per week it would have raised. Based on this evidence, the team approves one rule at the "warn" level and rejects the other. From then on, the pipeline flags every batch that breaks the approved rule, and the audit log records why the rule was adopted.

This makes the platform useful beyond the thesis. Banks are used to testing risk models on historical data before relying on them [34]; the platform applies the same discipline to data-quality controls, for hand-written rules as well as generated ones. For Avanade and Rabobank, it supports the onboarding and maintenance of data sources: a team can see which kinds of error its existing controls miss, review a small number of tested candidate rules, and test any new rule on its own historical data before it goes live. Because it runs with a local open-weight model, it can be tried on internal data without sending that data to an external service. It would still require internal security approval, data-specific adapters and integration work before production use, and the project does not promise measurable operational savings before user testing.

The scientific results of the thesis come from RQ1 to RQ3; the platform is the instrument that

produces them and the way in which they become usable. To keep the work manageable, the review screen will be simple, and the pipeline will be demonstrated on SEC data only.

9 Scope, feasibility and expected outputs

The project runs for nine months (Table 10). At the end of month 3, progress will be reviewed. The minimum scientific scope is the SEC/DQC evaluation for all three research questions, at least two model families, a controlled comparison of the selection methods, and an independently reviewed subset of real errors. HMDA is the preferred second domain, ESEF a related financial-reporting replication, and GLEIF an extension if time and reviewer capacity permit. If the programme becomes too large, GLEIF, then HMDA, then the depth of the ESEF analysis, and then the number of models will be reduced before the held-out evaluation is compromised.

Months	Work
1	Protocol, literature update, ethics request, cluster access, review guide, rule register and historical test engine
2–3	RQ1 and RQ2 on SEC/DQC; GLEIF test records; start of the D5 reviews; progress review at the end of month 3
4	RQ1 and RQ2 on HMDA and ESEF; data pipeline with the quality gate
5–6	RQ3 historical tests; completion of D5 and the reviewed alert sample; evidence cards and professional review
7	Analysis across datasets; robustness checks; demonstration to Avanade and Rabobank
8–9	Writing, documentation and contingency

Table 10: Nine-month plan.

Risk	Response
Generated rules add little, even with larger models	The thesis is a measurement; a clear negative result with its causes is informative.
Few reviewed real errors were visible in the original report	Weight D5 towards correction types that are more often data errors; report RQ3 with uncertainty intervals.
Reports that were never corrected are treated as correct	Use their alerts as a measure of review burden only; estimate false alarms from the reviewed sample.
The historical test uses information from the future	Store availability dates for every rule and model; run the time-bound form only after the model's release.
Building the pipeline takes time from the research	Keep the pipeline small and limited to SEC data; the scientific results come from RQ1 to RQ3.
Reviewers are not available in time	Request ethics approval in month one; report the limitation if no independent review is possible.

Table 11: Main risks and responses.

The expected outputs are a thesis that reports both successful and unsuccessful results, a reproducible experimental benchmark and selection procedure, a reviewed set of real errors subject to permissions, and a tested workbench and pipeline with documentation. An outcome in which generated rules do not provide reliable additional detections remains informative, provided that the controls, test data and limitations are reported transparently.

References

[1] L. Tran. Correspondence with the author, 4 September 2026. Unpublished; observations specific to his working context.

[2] Basel Committee on Banking Supervision. Principles for effective risk data aggregation and risk reporting (BCBS 239). January 2013. https://www.bis.org/publ/bcbs239.htm.

[3] European Central Bank. Guide on effective risk data aggregation and risk reporting. May 2024. https://doi.org/10.2866/081671.

[4] T. Rehberger, T. Hütter, L. Ehrlinger, and W. Wöß. Evaluating data quality tools: measurement capabilities and LLM integration. arXiv:2604.09163, 2026.

[5] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang. An empirical study of the non-determinism of ChatGPT in code generation. ACM Transactions on Software Engineering and Methodology, 34(2), 2025. https://doi.org/10.1145/3697010.

[6] A. Martin, E. C. de Almeida, O. Romero, and A. Queralt. How and why false denial constraints are discovered. Proceedings of the VLDB Endowment, 18(10):3477–3489, 2025. https://doi.org/10.14778/3748191.3748209.

[7] T. Oosterbaan. Data-quality controls with and without AI: what two pilot studies showed. Project working report, masterthesis_direction3_pilot, October 2026. Unpublished; numerical results reported by the author.

[8] T. Oosterbaan. Backtest feasibility check. Project working report, masterthesis_direction3_pilot, docs/backtest_feasibility.md, November 2026. Unpublished.

[9] S. Jäger and F. Biessmann. From data imputation to data cleaning: automated cleaning of tabular data improves downstream predictive performance. AISTATS, PMLR 238:3394–3402, 2024.

[10] Z. Abedjan et al. Detecting data errors: where are we and what needs to be done? Proceedings of the VLDB Endowment, 9(12):993–1004, 2016. https://doi.org/10.14778/2994509.2994518.

[11] M. Abdelaal, C. Hammacher, and H. Schöning. REIN: a comprehensive benchmark framework for data cleaning methods in ML pipelines. EDBT, 2023. https://doi.org/10.48786/edbt.2023.43.

[12] P. Jung, S. Jäger, N. Chandler, and F. Biessmann. Towards realistic error models for tabular data. ACM Journal of Data and Information Quality, 17(4), 2025. https://doi.org/10.1145/3774914.

[13] I.-C. Yeh. Default of credit card clients. UCI Machine Learning Repository, 2009. https://doi.org/10.24432/C55S3H.

[14] S. Moro, P. Rita, and P. Cortez. Bank marketing. UCI Machine Learning Repository, 2014. https://doi.org/10.24432/C5K306.

[15] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis. Wine quality. UCI Machine Learning Repository, 2009. https://doi.org/10.24432/C56S3T.

[16] M. Mahdavi et al. Raha: a configuration-free error detection system. SIGMOD, 865–882, 2019. https://doi.org/10.1145/3299869.3324956.

[17] XBRL US Data Quality Committee. DQC US rules, documentation and reference tests, version 31. Official repository, accessed November 2026. https://github.com/DataQualityCommittee/dqc_us_rules.

[18] B. Hui et al. Qwen2.5-Coder technical report. arXiv:2409.12186, 2024.

[19] A. Yang et al. Qwen3 technical report. arXiv:2505.09388, 2025.

[20] R. Alexander et al. Evaluating the decency and consistency of data validation tests generated by LLMs. arXiv:2310.01402, 2023.

[21] A. Akella, A. Kaul, K. Narayanam, and S. Mehta. Quality assessment of tabular data using large language models and code generation. EMNLP Industry Track, 2025. https://doi.org/10.18653/v1/2025.emnlp-industry.183.

[22] A.-C. Glock et al. Data quality rule generation with LLMs. arXiv:2609.06053, September 2026.

[23] J. Biggiogera et al. AuDQ: auditable data quality through agentic rule discovery. EMNLP Industry Track, 2026. https://aclanthology.org/2026.emnlp-industry.113/.

[24] A. Schneider. Template-guided rule generation and evaluation for data quality using large language models. Diploma thesis, TU Wien, 2025. https://doi.org/10.34726/hss.2025.135903.

[25] S. Siyam. Evaluating the effectiveness of LLMs and prompting techniques in generating data quality rules. Master's thesis, University of Jyväskylä, 2025. https://jyx.jyu.fi/handle/123456789/102965.

[26] S. Shankar et al. SPADE: synthesizing data quality assertions for large language model pipelines. Proceedings of the VLDB Endowment, 17(12), 2024. https://doi.org/10.14778/3685800.3685835.

[27] J. Song and Y. He. Auto-Validate: unsupervised data validation using data-domain patterns inferred from data lakes. SIGMOD, 2021. https://doi.org/10.1145/3448016.3457250.

[28] S. Redyuk, Z. Kaoudi, V. Markl, and S. Schelter. Automating data quality validation for dynamic data ingestion. EDBT, 2021.

[29] D. Tu et al. Auto-Validate by-History: auto-program data quality constraints to validate recurring data pipelines. KDD, 2023. https://doi.org/10.1145/3580305.3599776.

[30] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia. Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. CIDR, 2021.

[31] M. Armbrust et al. Delta Lake: high-performance ACID table storage over cloud object stores. Proceedings of the VLDB Endowment, 13(12):3411–3424, 2020. https://doi.org/10.14778/3415478.3415560.

[32] J. Tagliabue and C. Greco. Safe, untrusted, "proof-carrying" AI agents: toward the agentic lakehouse. IEEE Big Data Workshop on Secure and Safe AI Agents for Big Data Infrastructures, 2025. arXiv:2510.09567.

[33] T. Jin, Y. Zhu, and D. Kang. ELT-Bench: an end-to-end benchmark for evaluating AI agents on ELT pipelines. Proceedings of the VLDB Endowment, 19, 2025. arXiv:2504.04808.

[34] Basel Committee on Banking Supervision. Supervisory framework for the use of "backtesting" in conjunction with the internal models approach to market risk capital requirements. January 1996. https://www.bis.org/publ/bcbs22.htm.

[35] S. Joel, J. J. Wu, and F. H. Fard. A survey on LLM-based code generation for low-resource and domain-specific programming languages. ACM Transactions on Software Engineering and Methodology (accepted). arXiv:2410.03981.

[36] N. Jain et al. LiveCodeBench: holistic and contamination free evaluation of large language models for code. ICLR, 2025. arXiv:2403.07974.

[37] Team OLMo. 2 OLMo 2 Furious. arXiv:2501.00656, 2024.

[38] J. Liu et al. OLMoTrace: tracing language model outputs back to trillions of training tokens. ACL System Demonstrations, 2025. arXiv:2504.07096.

[39] Systems and methods for automatically ensuring data quality using LLMs. US Patent 12,632,447.

[40] Arelle. Open-source XBRL processor. https://arelle.org.

[41] US Securities and Exchange Commission. Financial Statement Data Sets, 2023q1 to 2025q2. https://www.sec.gov/data-research/sec-markets-data/financial-statement-data-sets.

[42] European Securities and Markets Authority. Enforcement of corporate reporting: ESEF reporting. https://www.esma.europa.eu/issuer-disclosure/enforcement-corporate-reporting.

[43] XBRL International. ESEF Data Quality Rules. https://esef-dqr.xbrl.org.

[44] XBRL International. ESEF DQR conformance suite. https://gitlab.xbrl.org/xii/dqr-conformance.

[45] FFIEC and CFPB. 2026 HMDA Filing Instructions Guide. https://ffiec.cfpb.gov/documentation/fig/2026/overview.

[46] Consumer Financial Protection Bureau. HMDA Platform, open-source filing and validation software. https://github.com/cfpb/hmda-platform.

[47] Global Legal Entity Identifier Foundation. Data quality checks and Data Quality Rule Setting, version 3.5. https://www.gleif.org/en/lei-data/gleif-data-quality-management/data-quality-checks.

[48] J. Cohen. A coefficient of agreement for nominal scales. Educational and Psychological Measurement, 20(1):37–46, 1960.

[49] University of Groningen. Hábrók cluster documentation: Apptainer containers. https://wiki.hpc.rug.nl/habrok/examples/apptainer.

[50] OpenAI. gpt-oss-120b and gpt-oss-20b model card. arXiv:2508.10925, 2025.

[51] M. Zaharia et al. Apache Spark: a unified engine for big data processing. Communications of the ACM, 59(11):56–65, 2016. https://doi.org/10.1145/2934664.

[52] Databricks. VACUUM, SQL language reference. https://docs.databricks.com/aws/sql/language-manual/delta-vacuum.